IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Does Learned Cost Guidance Improve Pre-XLA Graph Rewriting? A Target-Aligned, Paired Evaluation

SHWETANK SHRIJAN[1], SARVESH SUHAS KUTE[1], THOTA PENCHALA SANTHAN SAI[1], VIDHUR VAIBHAV B S[1], AND BHUVANESWARI M[1]
[1]Department of Computer Science, Vellore Institute of Technology, Vellore 632014, India (e-mail: shwetank.shrijan2024@vitstudent.ac.in; sarvesh.suhaskute2024@vitstudent.ac.in; thota.penchala2024@vitstudent.ac.in; vidhur.vaibhavbs2024@vitstudent.ac.in; m.bhuvaneswari@vit.ac.in)

Collaborating author: Bhuvaneswari M (e-mail: m.bhuvaneswari@vit.ac.in).

This work received no specific funding.

ABSTRACT Deep-learning compilers such as XLA lower tensor computation graphs to machine code, and graph-level rewrites such as operator fusion and algebraic simplification are applied before lowering to make the compiled program faster. Every such rewrite can be shown to preserve the output, but whether it reduces latency depends on the hardware, the tensor shapes and the optimizations the compiler already performs. Existing learned cost models predict program latency from large measured datasets to guide tuning and rewriting, while rule-based optimizers apply every legal rewrite. These models are usually trained on a different runtime from the one being optimized and are judged on whole-program latency, not on how well they rank single rewrites. We propose a target-aligned evaluation in which a graph neural network latency model is trained only on XLA-compiled timings and guides a greedy search over nine output-preserving rewrite rules; it is compared with four simple policies using 30 randomized, paired timing rounds and inspection of XLA's optimized programs. On 36 transformer workloads (3,699 timed graphs, 3,781 single-rewrite pairs), the model ranks graph latency well (Spearman correlation 0.91) but not single rewrites (0.00). Learned guidance is slightly slower than applying every legal rewrite (paired ratio 0.9987, $p = 0.008$) and no better than random selection, and only 27.4% of rewrites change the compiled program. Future work will train models on single-rewrite labels and extend the evaluation to GPUs and real exported models.

INDEX TERMS Compiler optimization, cost models, graph neural networks, graph rewriting, performance evaluation, XLA

I. INTRODUCTION

DEEP-LEARNING models are executed through compilers that lower a framework-level computation graph to hardware code through a stack of intermediate representations (Li et al., 2021). Nodes of the graph are tensor operations such as matrix multiplications, element-wise arithmetic, normalization and softmax, and edges are data dependencies. Before lowering, the graph can be rewritten: redundant operations removed, constants folded, producer–consumer pairs fused, or a whole attention block collapsed into one operator. Such rewrites are the first optimization stage of systems built on XLA, the compiler used by JAX and TensorFlow (He, 2023; Snider & Liang, 2023).

Every rewrite in such a system can be shown to preserve the output, but whether it makes the compiled program faster is a separate, empirical question. The answer depends on the hardware, the tensor shapes and, critically, on what the downstream compiler already does: XLA performs its own fusion, algebraic simplification and common-subexpression elimination (CSE), so a pre-XLA rewrite may be undone, duplicated or irrelevant. Applying every legal rewrite is therefore not guaranteed to help, and deciding which rewrites to apply is a natural target for learned guidance.

Learned compiler decisions are now well established. MLGO integrates learned inlining and register-allocation policies into LLVM (Trofin et al., 2021), and CompilerGym exposes compiler optimization decisions as reinforcement-learning environments (Cummins et al., 2022). For tensor programs, learned cost models predict the latency of candidate implementations so that a search procedure need not measure every candidate: TLP learns a cost model from the schedule primitives of tensor programs (Zhai et al., 2022), the one-shot tuner pre-trains a cost model to reduce on-device measurements (Ryu, Park, & Sung, 2022), and MetaSchedule and Hidet structure the search spaces such models explore

(Ding et al., 2023; Shao et al., 2022).

Graph-level rewriting has a parallel line of work. PET applies partially equivalent transformations and corrects their results automatically (Wang et al., 2021), Unity jointly optimizes algebraic transformations and parallelization (Unger et al., 2022), and TensorRight verifies tensor-graph rewrites formally (Arora et al., 2025). These systems use cost models or measured profiles to choose among the rewrites they can prove correct, and they establish that rewrite selection, not only rewrite correctness, determines the final performance.

These approaches leave two gaps for rewriting in front of an optimizing compiler. First, cost models are frequently trained on a different runtime from the one that executes the result, such as framework kernels or operator microbenchmarks, while the rewritten graph is compiled by XLA, which can compile the same operations differently. A model trained in one environment is not evidence that guidance works in another.

Second, cost models are mostly evaluated on whole-program latency. A rewrite policy, however, needs to rank single rewrites, whose effects are often a fraction of a percent. A model can rank whole graphs well and still be unable to tell a helpful rewrite from a harmful one. Small effects also require paired, repeated and randomized measurement before a policy can be called better or worse; without it, ties and slowdowns cannot be distinguished.

We address both gaps with a target-aligned, paired evaluation. Every training label and every evaluation measurement is the latency of an XLA-compiled program on the same CPU. A graph neural network (GNN) latency model is trained leave-one-architecture-out on 3,699 XLA-timed graphs and guides a greedy search over nine output-preserving rewrite rules. The learned (guided) policy is compared with applying every legal rewrite (all-safe), a fixed rule that never applies CSE, random selection of the same number of rewrites, and no rewriting, with 30 randomized timing rounds per variant, bootstrap confidence intervals, Wilcoxon signed-rank tests and Holm correction. A name-independent fingerprint of XLA's optimized HLO shows which rewrites change the program that is actually executed.

The approach is suited to the question because it isolates the two properties a learned policy needs, target-aligned labels and single-rewrite ranking, and measures them directly. Its advantage over the usual before-and-after comparison is that it can attribute a policy's result to its choices rather than to the number of rewrites it applies, and to compiled-program changes rather than to measurement noise. The study yields a clear negative result: the model ranks graphs well (Spearman $\rho = 0.91$) but not single rewrites ($\rho = 0.00$); guided rewriting is slightly slower than all-safe and indistinguishable from random selection; and only 27.4% of rewrites change XLA's compiled program.

Section II reviews 15 related studies. Section III presents the proposed system: problem formulation, architecture, modules, rewrite rules, latency model, policies, algorithms and evaluation protocol. Section IV describes the implementation, parameters and metrics and analyzes the results, including a comparison with three related studies and the threats to validity. Section V concludes and outlines future work.

II. RELATED WORKS

This section reviews 15 studies that inform the design of the proposed system. For each, we summarize the proposed solution, its methodology and tools, its results and the research gap that remains for pre-XLA rewrite selection.

A. XLA EFFICIENCY FOR NUMERICAL MODELS

He (2023) examines how efficiently the XLA compiler executes numerical models and where it falls short. The study compares XLA-compiled programs with alternative implementations and identifies the conditions under which compiler fusion translates into measurable speed. It argues that generated programs must be judged by execution, not by static characteristics.

The experiments use numerical computations of different operation types and problem sizes, run on CPU and GPU, and compare compiler-generated implementations with optimized reference implementations through measured execution time.

The analysis attributes the observed differences to the interaction between the computation, XLA's own transformations and the target hardware, and it discusses areas in which the compiler could improve.

Results and effectiveness: XLA delivers clear benefits for some numerical workloads but not uniformly; the gain depends on both the computation and the hardware, which supports execution-based evaluation of compiler transformations.

Research gap: It does not study rewrites applied before XLA or learned decisions about which rewrites to apply.

B. GRAPH-BASED CLUSTERING COST MODEL

Shen *et al.* (2026) propose a learned cost model for deep-learning compilers that exploits structural similarity between computation subgraphs. Subgraphs are grouped by their graph structure, and specialized predictors are built for the groups instead of one global model.

Graph information is turned into representations that can be compared across subgraphs, and clustering identifies groups with related characteristics. Separate prediction models then exploit the information associated with each group.

The method targets tensor-program optimization in a deep-learning compiler, where the cost model ranks candidate implementations during search, so that structurally different workloads do not have to share a single predictor.

Results and effectiveness: The authors report that cluster-specific prediction improves cost-model quality over a single model for their workloads, making structural grouping a useful design choice for learned compilers.

Research gap: It predicts program cost, not the effect of individual output-preserving rewrites in front of XLA.

C. TENSET PERFORMANCE DATASET

Zheng *et al.* (2021) introduce TenSet, a large dataset of measured tensor-program performance for training learned cost models. The goal is to make cost-model research independent of expensive on-device measurement.

Programs are generated from the search spaces of TVM's auto-scheduler for a range of deep-learning networks, and their execution times are measured on several CPU and GPU platforms. The dataset records each program together with its measured latency.

The authors use the dataset to train and compare cost-model architectures and to pre-train models that are then transferred to new tuning tasks.

Results and effectiveness: Pre-trained cost models reduce the search time needed to reach good tensor programs, showing the value of large, measured training sets.

Research gap: Its labels are TVM tensor programs; it does not provide labels for graph rewrites compiled by XLA.

D. FAMILYSEER SUBGRAPH SPECIALIZATION

Zhang *et al.* (2022) propose FamilySeer, which reduces auto-tuning cost by exploiting similarity between computation subgraphs. Related subgraphs are grouped into families that share tuning knowledge.

FamilySeer builds on a tensor-program auto-scheduler. Subgraphs are clustered into families by their computational similarity, and a cost model is trained per family so that measurements from one member inform the search for others.

The tuning budget is allocated across families, which reduces redundant hardware measurements for structurally similar subgraphs.

Results and effectiveness: The authors report that family-based cost models reach good tensor programs with less tuning time than a single shared cost model.

Research gap: Improved tuning efficiency is not the same objective as choosing which graph rewrites to apply before compilation.

E. TPUGRAPHS PERFORMANCE PREDICTION

Phothilimthana *et al.* (2023) present TpuGraphs, a dataset for predicting the performance of large tensor computation graphs under different compiler configurations.

The graphs are XLA HLO graphs of real machine-learning models, paired with layout and tile configurations and runtimes measured on Tensor Processing Units. Baseline GNN models are trained to rank configurations for a given graph.

The dataset frames compiler performance prediction as a ranking problem over configurations of the same computation, which is closely related to ranking rewrite candidates.

Results and effectiveness: GNN baselines learn useful configuration rankings, but the task remains difficult, showing that measured, configuration-level labels are required.

Research gap: It covers XLA configurations on TPUs, not pre-XLA graph rewrites on CPUs.

F. OPERATOR FUSION IN XLA

Snider and Liang (2023) analyze how XLA decides which operations to fuse and how these decisions affect performance.

They study XLA's fusion passes on JAX programs, inspect the fused kernels the compiler produces, and evaluate alternative fusion choices by measuring execution.

The analysis shows that the number of graph operations and the measured runtime are not monotonically related, because fusion benefits depend on the generated implementation.

Results and effectiveness: Changing fusion decisions can produce substantial speedups in specific cases, but XLA's own fusion already handles many patterns, so graph-level fusion must be judged after compilation.

Research gap: It does not evaluate learned guidance for rewrites applied before XLA.

G. GRAPHMAE MASKED GRAPH REPRESENTATIONS

Hou *et al.* (2022) propose GraphMAE, a self-supervised graph autoencoder that learns graph representations by reconstructing masked node features.

A GNN encoder processes graphs in which some node features are masked, and a decoder reconstructs them using a scaled cosine error; re-masking before decoding prevents trivial solutions.

The learned representations are evaluated on node classification, graph classification and molecular property prediction.

Results and effectiveness: Masked feature reconstruction gives competitive representations for downstream graph-learning tasks without task-specific labels.

Research gap: Generic graph representations do not encode tensor shapes or compiler behavior, which latency prediction requires.

H. MLIR COMPILER INFRASTRUCTURE

Lattner *et al.* (2021) introduce MLIR, a reusable compiler infrastructure that supports multiple abstraction levels in one framework.

Domain-specific operations are grouped into dialects, and computations are lowered progressively from high-level dialects to lower-level ones. The infrastructure is implemented in C++ within the LLVM project.

Analyses and transformations are written once and reused across dialects, so optimizations can run at whichever level holds the required information.

Results and effectiveness: MLIR reduces the cost of building domain-specific compilers and has become the basis of several machine-learning compilers, including parts of the XLA stack.

Research gap: It provides infrastructure for rewriting but not a decision procedure for which rewrites improve compiled latency.

I. EGG EQUALITY SATURATION

Willsey *et al.* (2021) present egg, a fast and extensible library for equality saturation with e-graphs.

Equality saturation keeps all equivalent forms of an expression in an e-graph instead of replacing one form with another, and a later extraction step picks the best form. egg introduces rebuilding to keep e-graphs efficient and e-class analyses to attach information to equivalence classes; it is implemented in Rust.

This removes the dependence of the result on the order in which rewrites are applied.

Results and effectiveness: egg is substantially faster than earlier e-graph implementations and supports several case studies in program optimization.

Research gap: The quality of the extracted program still depends on a cost function, which must match the target compiler.

J. TENSOR GRAPH SUPEROPTIMIZATION

Yang *et al.* (2021) apply equality saturation to tensor computation graphs to find faster equivalent graphs.

Tensor rewrite rules are applied to an e-graph built with egg, and the best graph is extracted with integer linear programming under a cost model of operator runtimes.

Keeping all equivalent graphs avoids the phase-ordering problem of greedy sequential rewriting.

Results and effectiveness: The approach finds graphs that are as good as or better than those of sequential superoptimizers, in much less optimization time.

Research gap: Its extraction relies on an operator cost model that is not calibrated against the downstream compiler's fused output.

K. DNNFUSION OPERATOR FUSION

Niu *et al.* (2021) propose DNNFusion, which fuses operators more aggressively than fixed fusion patterns.

Operators are classified by their input–output mapping types, and fusion opportunities are found from these classes; mathematical-property-based graph rewriting simplifies the graph before a fusion plan is generated, and fused code is generated for mobile devices.

Profiling information is used to decide whether a fusion is worthwhile, so legality alone does not decide fusion.

Results and effectiveness: DNNFusion reduces the number of fused layers and speeds up inference on mobile CPUs and GPUs compared with established mobile frameworks.

Research gap: It generates its own fused kernels; it does not study rewrites whose output is compiled again by XLA.

L. DIETCODE DYNAMIC-SHAPE OPTIMIZATION

Zheng *et al.* (2022) address auto-scheduling for tensor programs whose shapes vary at run time.

DietCode builds a shape-generic search space from micro-kernels and a cost model that is shared across shapes, so tuning one shape informs others. It is built on TVM and evaluated on GPUs.

This avoids tuning every shape independently.

Results and effectiveness: DietCode reduces tuning time substantially for dynamic-shape workloads while keeping performance competitive with per-shape tuning.

Research gap: Our latency model also uses shape features, but rewrite selection under varying shapes is not studied.

M. ROLLER TENSOR COMPILATION

Zhu *et al.* (2022) propose ROLLER, which constructs tensor programs from hardware-aligned tiles instead of searching a large space.

The rTile abstraction aligns tile shapes with hardware characteristics, programs are built recursively from rTiles, and a micro-performance model estimates their cost without measuring every candidate. ROLLER is evaluated on GPUs and other accelerators.

This shifts compilation from search to construction.

Results and effectiveness: ROLLER produces competitive kernels with compilation times of seconds instead of hours.

Research gap: Its tile-level model works below the graph level and does not decide graph rewrites.

N. WELDER MEMORY-ACCESS SCHEDULING

Shi *et al.* (2023) propose Welder, which schedules deep-learning memory access across operators.

A tile-graph represents operators and their tiles together, so that intermediate data can be kept in faster memory levels across operator boundaries. The scheduler optimizes data reuse through the memory hierarchy, and kernels are generated for GPUs.

This treats operator fusion as a memory-scheduling problem.

Results and effectiveness: Welder reduces memory traffic and improves end-to-end performance over existing compilers and vendor libraries on several models.

Research gap: Its memory model is not part of graph-level rewrite selection before XLA.

O. TENSORIR TENSORIZED PROGRAMS

Feng *et al.* (2023) introduce TensorIR, an abstraction for automatic tensorized program optimization.

TensorIR adds a block construct that isolates tensorized computations, together with scheduling primitives and a search procedure; it is implemented in Apache TVM and targets hardware tensor instructions on CPUs and GPUs.

The abstraction lets the compiler reason about tensor instructions that a graph-level representation cannot express.

Results and effectiveness: TensorIR reaches performance competitive with vendor libraries on several hardware platforms.

Research gap: It optimizes below the graph level; graph-rewrite decisions remain outside its scope.

Across these studies, three requirements recur: features relevant to the decision being predicted, labels from the target that executes the result, and validation against the compiled program. Further related systems are discussed where they are used: transfer-tuning (Gibson & Cano, 2022) for reusing

schedules, and FlashAttention (Dao, 2024; Dao, Fu, Ermon, Rudra, & Ré, 2022) for attention fusion.

III. PROPOSED WORK AND SYSTEM

A. PROBLEM FORMULATION

A workload is a directed acyclic graph $G = (V, E)$ whose nodes are tensor operations and whose edges are data dependencies, together with the tensor sizes: batch B, sequence length S and model width D. A rewrite rule maps a matching site of G to an equivalent subgraph; applying one rule at one site gives a candidate $c(G)$. Let $\mathcal{R}^*(G)$ be every graph reachable by sequences of legal rewrites and $T_{\mathrm{XLA}}(G)$ the latency of the program XLA compiles from G. The optimization problem is

$$G^* = \arg\min_{G' \in \mathcal{R}^*(G)} T_{\mathrm{XLA}}(G') \quad \text{subject to} \quad y(G') \approx y(G), \tag{1}$$

where $y(\cdot)$ is the output. Output agreement is checked elementwise against the original,

$$\max_j \left| y_j(G') - y_j(G) \right| \le 10^{-4} \max_j \left| y_j(G) \right|. \tag{2}$$

This is a purely relative check: zero-valued reference outputs require exact equality, and near-zero values can make the criterion overly strict. The reported check is therefore an empirical check on the tested input, not a proof of equivalence for all inputs.

Because $\mathcal{R}^*(G)$ is large and measuring T_{XLA} requires compilation, a policy chooses rewrites one at a time. A learned policy uses a latency model $\hat{\ell}(G) \approx \log T_{\mathrm{XLA}}(G)$. The study asks whether such a model, trained on XLA timings, ranks single rewrites well enough to choose better than simple policies, and which rewrites change the compiled program at all.

B. SYSTEM ARCHITECTURE

Fig. 1 shows the architecture. Stage A holds the inputs: 36 workloads, nine rewrite rules and random rewrite trajectories that generate partially rewritten graphs. Stage B compiles every graph with XLA, times it, and trains the latency model on these target-aligned labels. Stage C compiles the variants produced by five policies, times them under a paired randomized protocol, compares them statistically, and fingerprints XLA's optimized program for each.

The stages are separated so that each claim is supported by the right evidence. Prediction quality is measured on architectures the model never saw (Stage B); policy quality is measured by execution of compiled programs (Stage C); and whether a rewrite matters to XLA at all is measured by comparing optimized programs. The guided policy is the only component that uses the model, so its result can be compared directly with policies that do not.

C. MODULES AND THEIR FUNCTIONS

Table 1 lists the modules of the implementation. The graph IR, rewrite rules and JAX lowering form the compiler front end; the latency model and guided search form the learned component; and the study driver, statistics, HLO and environment modules form the evaluation. Each module is used by exactly one stage of Fig. 1, which keeps the evidence for each result traceable to the code that produced it.

TABLE 1. Modules of the implementation and their functions.

Module	Function
`ir/graph.py`	Graph IR: operations, dependencies, operand order, tensor sizes
`search/rewrites.py`	Nine rewrite rules; enumeration and application of legal rewrites; all-safe and random policies
`search/neuro_symbolic.py`	Latency GNN, training, guided search with decision counts
`utils/jax_executor.py`	Lowering of any IR graph to a JAX function for XLA
`utils/hlo.py`	Canonical form, fingerprint and diff of optimized HLO
`utils/stats.py`	Paired speedup, bootstrap CI, Wilcoxon test, Holm correction
`utils/environment.py`	Records CPU, OS, threads, library versions and commit
`benchmarks/xla_rewrite_study.py`	Data collection, training, policy timing, HLO report
`tools/make_xla_tables.py`	Regenerates every table and the raw-sample CSV

TABLE 2. Output-preserving rewrite rules.

Rule	Rewrite
eliminate-identity	$\mathrm{Identity}(x) \Rightarrow x$
fold-constant	operation on constants only $\Rightarrow$ constant
merge-scales	$\mathrm{Scale}_b(\mathrm{Scale}_a(x)) \Rightarrow \mathrm{Scale}_{ab}(x)$
cse	duplicate operation on the same inputs $\Rightarrow$ shared result
double-transpose	$(A^\top)^\top \Rightarrow A$
fold-transpose	$A \cdot B^\top \Rightarrow$ matmul with transposed right operand
fuse-pair	MatMul+Add$\rightarrow$GEMM; GEMM or MatMul with GELU, ReLU or SiLU; Add with LayerNorm or RMSNorm; Scale+Softmax
fuse-attention	Q, K, V projections, $QK^\top$, scale, softmax, $AV \Rightarrow$ one attention node
remove-dead	unused non-output operation $\Rightarrow$ removed

D. GRAPH REPRESENTATION AND REWRITE RULES

Each node of the IR is lowered to the corresponding `jax.numpy` operation and the resulting function is compiled with `jax.jit`; fused IR nodes are lowered to their mathematical definition, so XLA remains free to apply its own fusion to every variant. Table 2 lists the nine rewrite rules. Each rule enumerates every legal site in a graph and rewrites one site on a copy; a rewrite that would create a cycle is discarded.

Fig. 2 illustrates the rules on a feed-forward fragment with injected redundancy. Identity elimination and scale merging remove three nodes, and fusing the matrix multiplication with its bias addition produces one GEMM operator. Each step preserves the output; whether it changes XLA's compiled pro-

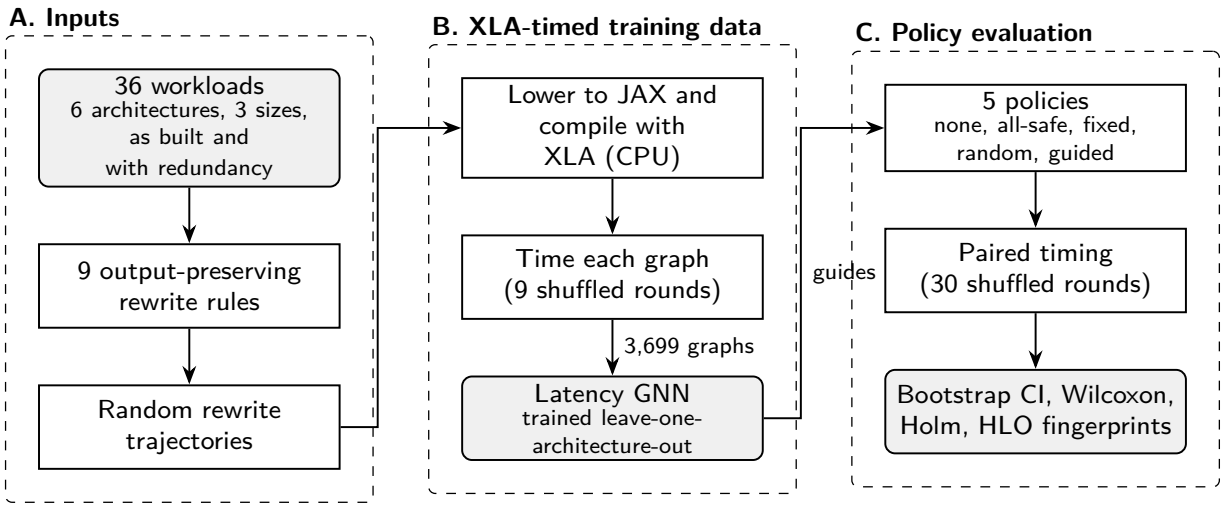


FIGURE 1. Architecture of the proposed system. (A) Rewrite rules are applied to 36 workloads. (B) Every graph reached along random rewrite trajectories is compiled with XLA and timed; these timings train the latency model, one architecture held out at a time. (C) Five policies are compiled, timed in 30 shuffled paired rounds and compared statistically, and their optimized HLO is fingerprinted.

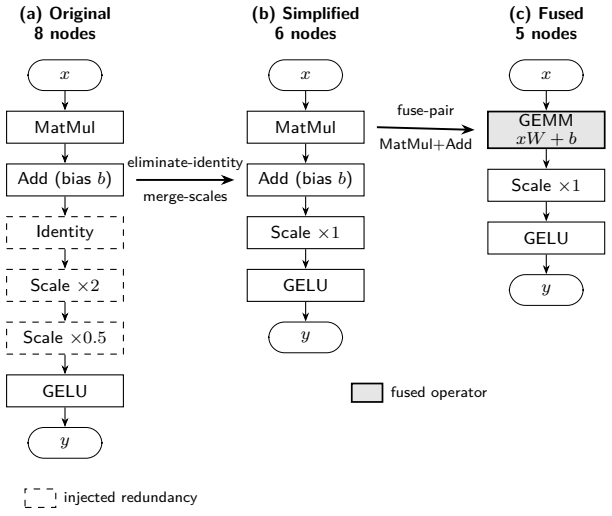


FIGURE 2. Rewrite example. (a) A linear layer with GELU and injected redundancy (dashed). (b) After identity elimination and scale merging. (c) After fusing MatMul and the bias Add into one GEMM.

gram is measured separately (Section IV-H). The attention-fusion rule targets the same pattern as FlashAttention (Dao, 2024; Dao et al., 2022) but is lowered to a composed implementation.

E. LATENCY MODEL

The latency model (Fig. 3) is an additive message-passing GNN that predicts the logarithm of XLA latency in microseconds. Each node v has 36 features x_v: a one-hot operation type over 32 types; in-degree and out-degree, each capped at 8 and divided by 8; a per-operation latency prior from a calibrated cost table, divided by 150; and the node's topological level divided by the graph depth. The graph has 13 features g: $\log|V|/5$, depth/60, maximum and mean level width (/30, /8), $|V|$/depth/8, the total-to-critical-path ratio of the prior latencies /4, log total prior latency /12, the parallelizable fraction of the prior latency, and the size features $\log B/3$, $\log S/6$, $\log D/6$, $\log(BSD)/14$ and $\log(1+\text{prior})/8$.

The features are projected to a 64-dimensional state,

$$h_v^{(0)} = \mathrm{ReLU}\big(W_{\text{in}}[x_v \| g]\big), \tag{3}$$

and two message-passing rounds update it with forward and backward messages,

$$\begin{aligned} h_v^{(k+1)} = h_v^{(k)} &+ \mathrm{ReLU}\big(W_k^f[h_v^{(k)} \| \bar{h}_v^{\text{pred}}]\big) \\ &+ \mathrm{ReLU}\big(W_k^b[h_v^{(k)} \| \bar{h}_v^{\text{succ}}]\big), \end{aligned} \tag{4}$$

where $\bar{h}_v^{\text{pred}}$ and $\bar{h}_v^{\text{succ}}$ are the means of $h^{(k)}$ over the predecessors and successors of v. A two-layer MLP (64–64–1) with a softplus output gives a non-negative time t_v per node, and the graph prediction is

$$\hat{\ell}(G) = \log\Big(\sum_{v\in V} t_v + 10^{-6}\Big) + s, \tag{5}$$

with a learned offset s. The additive form lets one model generalize across architectures that reuse the same operations. The model has 40,450 parameters and is trained with the smooth-L_1 loss on log latency,

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \mathrm{smooth}_\beta\big(\hat{\ell}(G_i) - \log T_i\big), \quad \beta = 0.1, \tag{6}$$

using Adam (learning rate 4×10^{-3}, weight decay 10^{-6}), cosine learning-rate decay and 400 full-batch epochs, with seed 0 and s initialized to the mean label minus 2. The predicted speedup of a rewrite c is

$$\hat{\sigma}(c) = \exp\big(\hat{\ell}(G) - \hat{\ell}(c(G))\big). \tag{7}$$

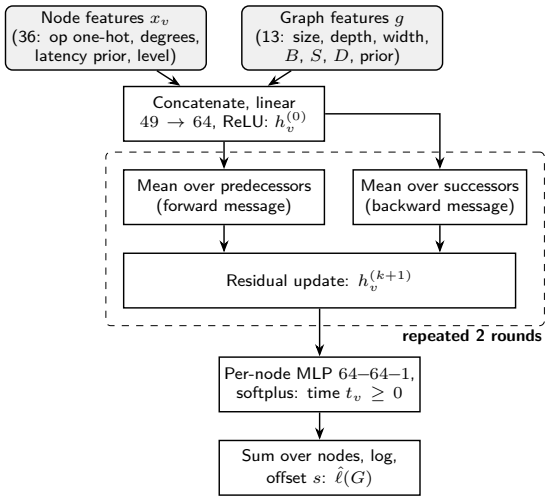


FIGURE 3. **Architecture of the latency GNN. Node and graph features are projected to a 64-dimensional state, updated by two rounds of forward and backward message passing, and mapped to non-negative per-node times whose sum gives the predicted log latency.**

F. REWRITE POLICIES AND GUIDED SEARCH

A policy decides which legal rewrites to apply. We compare five: *none* (the original graph); *all-safe*, which repeatedly applies the first legal rewrite in the rule order of Table 2 until none remains; a *fixed rule* that is all-safe with CSE disabled; *guided* search scored by the latency model; and *random (same count)*, which applies, uniformly at random among legal rewrites, as many rewrites as the guided policy applied to that workload. The last policy separates the effect of *which* rewrites are chosen from the effect of *how many*.

The guided policy (Fig. 4, Algorithm 1) enumerates every legal one-step rewrite of the current graph, predicts its log latency, and accepts the best candidate c if

$$\begin{aligned} &\hat{\ell}(c(G)) < \hat{\ell}(G) - \log \tau \\ &\vee \ \big(\hat{\ell}(c(G)) \leq \hat{\ell}(G) + \epsilon \ \wedge \ |c(G)| < |G|\big), \end{aligned} \tag{8}$$

where $\tau \geq 1$ is the required predicted speedup ($\tau = 1$ in the policy runs), $\epsilon = 0.003$ is a tie tolerance in log units and $|G|$ is the node count. The search repeats until no candidate is accepted. The second clause accepts any node-reducing rewrite predicted to be within 0.3% of the current latency; Section IV-F shows that it dominates the policy's behavior.

G. TARGET-ALIGNED DATA COLLECTION AND TRAINING

Algorithm 2 collects the training data. For each workload, two random rewrite trajectories of up to ten steps are generated. At each step the current graph and up to six of its legal one-step rewrites are compiled with XLA and timed together in nine rounds, with the program order shuffled every round; each graph's label is its median time, and a graph already timed is not timed again. Each step also yields measured pairs (graph, rewrite, rewritten graph) with speedup $T_{\text{before}}/T_{\text{after}}$. The model is trained leave-one-architecture-out: for each architecture, it is trained on the other five and evaluated on the held-out one, and the guided policy for a workload always uses the model that never saw its architecture.

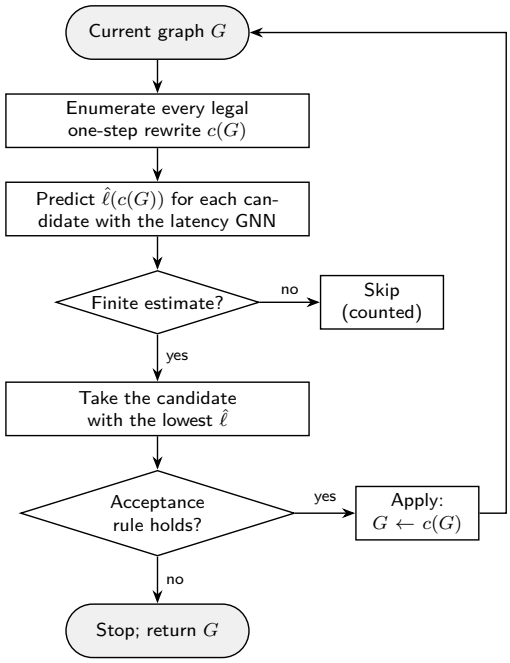


FIGURE 4. **Flow of the guided policy. Candidates without a finite estimate are skipped and counted; the search stops when the best remaining candidate fails the acceptance rule** (8).

Algorithm 1 Guided rewrite search

Require: graph G; latency model $\hat{\ell}$; threshold τ; tolerance ϵ
Ensure: rewritten graph G; decision counts

$b \leftarrow \hat{\ell}(G)$
loop
  $C \leftarrow \{c(G)\}$ for every legal rule site; discard cyclic results
  if $C = \emptyset$ **then**
    break
  end if
  score every $G' \in C$ with $\hat{\ell}$; count non-finite scores and skip them
  $G' \leftarrow \arg\min_{G' \in C} \hat{\ell}(G')$
  if G' satisfies (8) with respect to G **then**
    $G \leftarrow G', b \leftarrow \hat{\ell}(G')$
  else
    break
  end if
end loop
return G

H. EVALUATION PROTOCOL

Fig. 5 and Algorithm 3 give the evaluation protocol. For each workload, the five policy variants are compiled once and checked against (2). They are timed in $R = 30$ rounds; in each round every variant is timed once, in a freshly shuffled order, and one sample is the mean of enough blocking calls

Algorithm 2 XLA-timed data collection and training

Require: workloads $\mathcal{W}$; rules $\mathcal{R}$; 2 trajectories, 10 steps, 6 candidates, 9 rounds
Ensure: timed graphs $\mathcal{D}$; measured pairs $\mathcal{P}$; one model per architecture

for all $w \in \mathcal{W}$ and each trajectory **do**
  $G \leftarrow G_w$
  for step $= 1$ **to** 10 **do**
    $C \leftarrow$ up to 6 random legal rewrites of G
    compile the untimed graphs of $\{G\} \cup C$ with XLA; time them in 9 shuffled rounds; add medians to $\mathcal{D}$
    add $(G, c, c(G), T(G)/T(c(G)))$ to $\mathcal{P}$ for $c(G) \in C$
    $G \leftarrow$ a random member of C
  end for
end for
for all architectures a **do**
  train $\hat{\ell}_a$ on graphs of $\mathcal{D}$ not from a by (6); evaluate on graphs and pairs from a
end for

to last at least 2 ms. Because all variants are sampled in every round, round r gives a paired speedup

$$\sigma_r = T_r^{\text{none}} / T_r^{\text{variant}}, \qquad r = 1, \ldots, R. \tag{9}$$

For each workload and policy we report the median of σ_r and its 95% percentile-bootstrap interval $[\sigma_{\text{lo}}, \sigma_{\text{hi}}]$ (2,000 resamples), and classify the variant as

$$\text{verdict} = \begin{cases} \text{faster}, & \sigma_{\text{lo}} > 1, \\ \text{slower}, & \sigma_{\text{hi}} < 1, \\ \text{indistinguishable}, & \text{otherwise}. \end{cases} \tag{10}$$

The two-sided Wilcoxon signed-rank test is applied to $d_r = \log \sigma_r$ with statistic

$$W^+ = \sum_{r:\, d_r > 0} \text{rank}\big(|d_r|\big), \tag{11}$$

and across the $m = 36$ workloads Holm's procedure rejects the hypotheses with the k smallest p-values, where k is the largest index such that

$$p_{(j)} \le \frac{\alpha}{m - j + 1} \quad \text{for all } j \le k, \qquad \alpha = 0.05. \tag{12}$$

Guided and all-safe are also compared directly, per workload and across workloads, by applying the Wilcoxon test to the 36 per-workload median log-ratios.

For each compiled program, the optimized HLO text is canonicalized: source-location tables and metadata are removed, and every computation, instruction and parameter is renamed in order of first appearance. Two programs have the same fingerprint exactly when XLA generated the same code. Fingerprints are recorded for every measured pair and every policy variant.

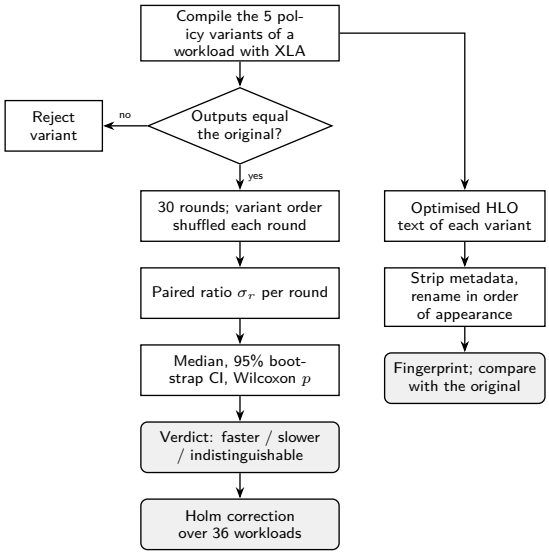


FIGURE 5. **Evaluation protocol. Left: correctness check, paired randomized timing and statistical verdicts. Right: fingerprinting of XLA's optimized HLO to detect whether a variant changes the compiled program.**

Algorithm 3 Paired randomized timing of policies

Require: workload G; policies Π; rounds $R = 30$
Ensure: raw samples; verdicts; fingerprints

compile $\pi(G)$ for every $\pi \in \Pi$; reject variants failing (2)
choose n calls per sample so that one sample lasts ≥ 2 ms
for $r = 1$ **to** R **do**
  for all π in a freshly shuffled order of Π **do**
    record T_r^{π}, the mean of n blocking calls
  end for
end for
compute (9), the bootstrap interval, (10) and (11)
fingerprint the optimized HLO of every variant

IV. IMPLEMENTATION AND RESULT ANALYSIS

A. TOOLS, CONFIGURATION AND SETUP

The system is implemented in Python. JAX (Bradbury et al., 2018) lowers each graph to a function that XLA compiles for the CPU, and PyTorch (Ansel et al., 2024) implements and trains the latency GNN; NumPy, SciPy and NetworkX provide arrays, statistics and graph utilities. All results come from a single run on one machine (Table 3); every result file records this environment automatically, together with the code commit. No GPU was used.

The experiment is reproduced from the repository with the commands below. The study writes every timing sample, measured pair, fold split, model setting and environment record to one results file, and the table script regenerates every table of this section from it; the test suite checks output equivalence of every policy and the statistics and fingerprint code.

```
python3.12 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python benchmarks/xla_rewrite_study.py
.venv/bin/python tools/make_xla_tables.py
.venv/bin/python -m unittest discover -s tests
```

TABLE 3. Experimental environment of every reported result.

Item	Value
CPU	Apple M3 Pro (arm64), 12 cores
OS	macOS 27.0 (Darwin 27.0.0)
Python	3.12.14
JAX / jaxlib	0.12.0.1 / 0.12.0.1 (XLA CPU backend)
PyTorch	2.14.1 (6 intra-op threads)
NumPy, SciPy, NetworkX	2.5.4, 1.18.1, 3.7
Thread settings	library defaults; no `XLA_FLAGS`
Code	commit `d081a56`

TABLE 4. Parameter setup of the full run.

Parameter	Value
Workloads	6 architectures $\times$ 3 sizes $\times$ 2 versions = 36
Sizes (B, S, D)	(4, 64, 64), (4, 128, 128), (8, 256, 256)
Injected redundancy	up to 8 identities or $\text{Scale}_a/\text{Scale}_{1/a}$ pairs, seeded
Trajectories	2 per workload, up to 10 steps, 6 candidates per step
Label timing	9 shuffled rounds per group; median
Policy timing	30 shuffled rounds; sample $\geq$ 2 ms (1–5 calls)
GNN	64 hidden, 2 rounds, 40,450 parameters
Training	400 epochs, Adam, lr 4×10^{-3}, weight decay 10^{-6}, smooth-L_1 $\beta = 0.1$, seed 0
Acceptance	$\tau = 1$ (also 1.02, 1.05 evaluated), $\epsilon = 0.003$
Outcome band	beneficial ≥ 1.01, harmful ≤ 0.99
Statistics	2,000 bootstrap resamples, 95% CI, Holm $\alpha = 0.05$
Correctness	relative tolerance 10^{-4}

B. IMPLEMENTATION TOOL

Fig. 6 shows the study running in a terminal in its quick mode, which uses the smallest size, one trajectory per workload and five timing rounds to check the pipeline in about 30 s. The output lists the held-out evaluation of each architecture's model and the per-workload policy verdicts. The full run used for the results differs only in its configuration (Table 4).

C. PARAMETER SETUP

Table 4 lists the parameters of the full run. We use six transformer-style architectures, each with two blocks: Post-LN, Pre-LN, Post-LN with ReLU feed-forward, LLaMA-style (RMSNorm, SwiGLU), mixture-of-experts with four experts, and multi-head attention with four heads. Original latencies range from 398.5 µs to 19,542.7 µs per call, and the median graph has 47 nodes.

D. PERFORMANCE METRICS

We evaluate seven metrics:

1) *Latency and speedup*: per-call latency in microseconds and the paired speedup (9) over no rewriting.
2) *Prediction error*: mean absolute error (MAE, in microseconds) and mean absolute percentage error (MAPE) of predicted graph latency, and MAE of predicted single-rewrite speedup (7).
3) *Ranking quality*: Spearman correlation ρ between predicted and measured latency or speedup, i.e., the Pearson correlation of their ranks.
4) *Decision accuracy*: counts of accepted and rejected rewrites that were measured to be beneficial, neutral or harmful.
5) *Compiled-program change*: the fraction of rewrites whose optimized HLO fingerprint differs from the original's.
6) *Graph reduction*: the percentage of IR nodes removed.
7) *Compilation overhead*: XLA compile time in milliseconds, optimized HLO instruction and fusion counts, and XLA scratch memory in KiB.

TABLE 5. Latency model on held-out architectures (labels: XLA latency). Latency errors in microseconds; ρ: Spearman correlation; "near 1": pairs predicted within $\pm$5% of 1.

	Graph latency			Single-rewrite speedup		
Held-out	MAE (µs)	MAPE	ρ	MAE	ρ	ρ near 1
Post-LN	210.6	5.9%	0.90	0.037	0.03	0.03
Pre-LN	495.7	18.7%	0.90	0.052	−0.02	−0.07
Post-LN+ReLU	337.4	11.8%	0.92	0.040	0.05	0.08
LLaMA-style	263.8	13.0%	0.88	0.038	−0.06	−0.07
MoE	785.4	20.0%	0.90	0.032	−0.03	−0.02
Multi-head	3,539.6	54.9%	0.94	0.035	0.03	−0.02
Mean	938.8	20.8%	0.91	0.039	0.00	−0.01

E. PREDICTION ERROR AND RANKING QUALITY

Table 5 reports the latency model on held-out architectures. Graph latency is ranked well: the mean Spearman correlation is 0.91, with a MAPE of 20.8%. The MAE of 938.8 µs is driven by the multi-head architecture (3,539.6 µs, 54.9%), whose four-way head split differs most from the training architectures.

Ranking single rewrites is a different task, and the model fails at it. The mean correlation between predicted and measured pair speedups is 0.00; the predicted direction agrees with the measured one for 48.8% of pairs; and the correlation restricted to pairs predicted within $\pm 5\%$ of 1, where decisions are made, is -0.01. The reason is scale: 52.9% of measured pairs change latency by less than 1%, far below the model's 20.8% error on whole graphs.

F. DECISION ACCURACY

Table 6 compares each pair's accept–reject decision with its measured outcome. With the rule used by the guided policy ($\tau = 1$), 2,113 of 3,781 pairs are accepted; 25.7% of accepted pairs are beneficial, against a base rate of 26.8% among all pairs, so acceptance is no better than chance. The policy accepts 425 harmful rewrites and rejects 468 beneficial ones. Raising τ to 1.05 barely changes the counts because the tie clause of (8) accepts most node-reducing rewrites regardless of τ; with the threshold alone, $\tau = 1.05$ accepts only 94

Terminal | compiler-optimization-for-llm (branch integrated-final, commit d081a56)

```
$ python benchmarks/xla_rewrite_study.py --quick --out results/quick.json
jax 0.12.0.1 cpu [CpuDevice(id=0)]
12 workloads; collecting XLA-timed rewrite data
  data Post-LN Transformer | B2 S32 D32 | base  graphs  7 pairs  6 (1s)
  ...
held-out Post-LN Transformer  latency MAE  18.8 us MAPE  21.1% | pair rho -0.50 wrong-accept 6 missed 1
held-out Pre-LN Transformer  latency MAE  12.8 us MAPE  13.8% | pair rho -0.52 wrong-accept 5 missed 0
held-out Post-LN + ReLU  latency MAE  25.4 us MAPE  29.3% | pair rho 0.20 wrong-accept 4 missed 0
held-out LLaMA-style  latency MAE  10.3 us MAPE  10.7% | pair rho -0.15 wrong-accept 6 missed 0
held-out MoE Transformer  latency MAE  7.9 us MAPE  6.3% | pair rho 0.29 wrong-accept 1 missed 0
held-out Multi-head Transformer  latency MAE  93.7 us MAPE  40.7% | pair rho -0.36 wrong-accept 4 missed 0
timing 5 policies x 12 workloads x 5 rounds
  Post-LN Transformer | B2 S32 D32 | base  all_safe 1.088i guided 1.022i random_same 1.010i fixed_no_cse 1.013i  match=True
  Post-LN Transformer | B2 S32 D32 | injected  all_safe 1.084i guided 1.109f random_same 1.024i fixed_no_cse 1.016i  match=True
  Pre-LN Transformer | B2 S32 D32 | base  all_safe 0.994i guided 1.006i random_same 0.997i fixed_no_cse 1.004i  match=True
  Pre-LN Transformer | B2 S32 D32 | injected  all_safe 1.027i guided 1.026i random_same 1.021i fixed_no_cse 1.032i  match=True
  ...
 "all_outputs_match": true
wrote results/quick.json
```

FIGURE 6. Console output of the study script in quick mode (excerpt). Each held-out line reports the latency model on an architecture it never saw; each workload line reports the paired speedup of every policy over no rewriting with its verdict (f: faster, s: slower, i: indistinguishable).

TABLE 6. Accept–reject decisions against measured outcomes, pooled over held-out folds (3,781 pairs). B, N, H: beneficial (≥ 1.01), neutral, harmful (≤ 0.99).

		Accepted			Rejected		
Rule	τ	B	N	H	B	N	H
Guided, Eq. (8)	1.00	544	1,144	425	468	857	343
Guided, Eq. (8)	1.05	536	1,137	415	476	864	353
Threshold only	1.00	491	1,067	366	521	934	402
Threshold only	1.05	30	42	22	982	1,959	746

TABLE 7. Policies compared with no rewriting over 36 workloads. Verdicts use the 95% bootstrap interval of the paired speedup; "Holm": significant after Holm correction; "Same HLO": optimized XLA program identical to the original's.

Policy	Suite median [95% CI]	Faster	Slower	Indist.	Holm	Same HLO
All-safe	1.0015 [1.0006, 1.0045]	13	0	23	11	18
Fixed (no CSE)	1.0024 [1.0006, 1.0049]	12	0	24	12	18
Random (same #)	1.0017 [1.0002, 1.0030]	7	2	27	4	10
Guided	1.0006 [0.9993, 1.0027]	9	2	25	7	11

pairs, of which 30 are beneficial. During the policy runs the guided search scored 3,993 candidates; none lacked a finite estimate, and every rejection at termination was a candidate predicted to be slower. The guided policy applied 9.8 rewrites per workload on average, against 15.2 for all-safe.

G. LATENCY AND SPEEDUP

Table 7 and Fig. 7 compare the policies with the original graphs. All policies are within 0.3% of no rewriting at the suite median. All-safe and the fixed rule are never significantly slower than the original; guided and random are slower on two workloads each. The gains are concentrated in the injected-redundancy versions: on as-built graphs the median speedup is 1.0006 for all-safe and 0.9989 for guided, and all-safe is faster on one of 18 workloads; with injected redundancy the medians are 1.0167 and 1.0029, and all-safe is faster on 12 of 18. The fixed rule behaves like all-safe because CSE applies on only two workloads.

Compared directly, guided is slower than all-safe: the median per-workload ratio (all-safe time over guided time) is 0.9987 with 95% interval [0.9981, 0.9998], and the Wilcoxon test across workloads gives $p = 0.008$. Guided is slower on 7 workloads, faster on none and indistinguishable on 29; on 9 workloads both policies produce the same optimized program. The deficit comes from rewrites guided declines: across the injected workloads all-safe merges 56 scale pairs and guided 23, and on the injected LLaMA-style workload at $S = D = 128$ all-safe applies five scale merges and gains 7.2% while guided applies none and gains nothing (0.999). Compared directly with random selection of the same number of rewrites, guided is indistinguishable: median ratio 0.9994 [0.9982, 1.0006], Wilcoxon $p = 0.72$, faster on 5 workloads and slower on 5. Guided's result is therefore explained by how many rewrites it applies, not by which ones it chooses.

H. COMPILED-PROGRAM CHANGE

Fig. 8 shows the measured speedup of single rewrites by rule, split by whether the rewrite changed XLA's optimized program. Only 27.4% of rewrites change it. Constant folding and dead-code removal never do (0 of 127 and 0 of 85): XLA performs both itself. Identity elimination changes the program in 44 of 491 cases and producer–consumer fusion in 507 of 2,313. Scale merging almost always changes it (319 of 343) and has the largest median effect (1.006), which is why the injected scale pairs are where the policies gain.

The policy variants confirm this. All-safe produces exactly the original optimized program on 18 of 36 workloads

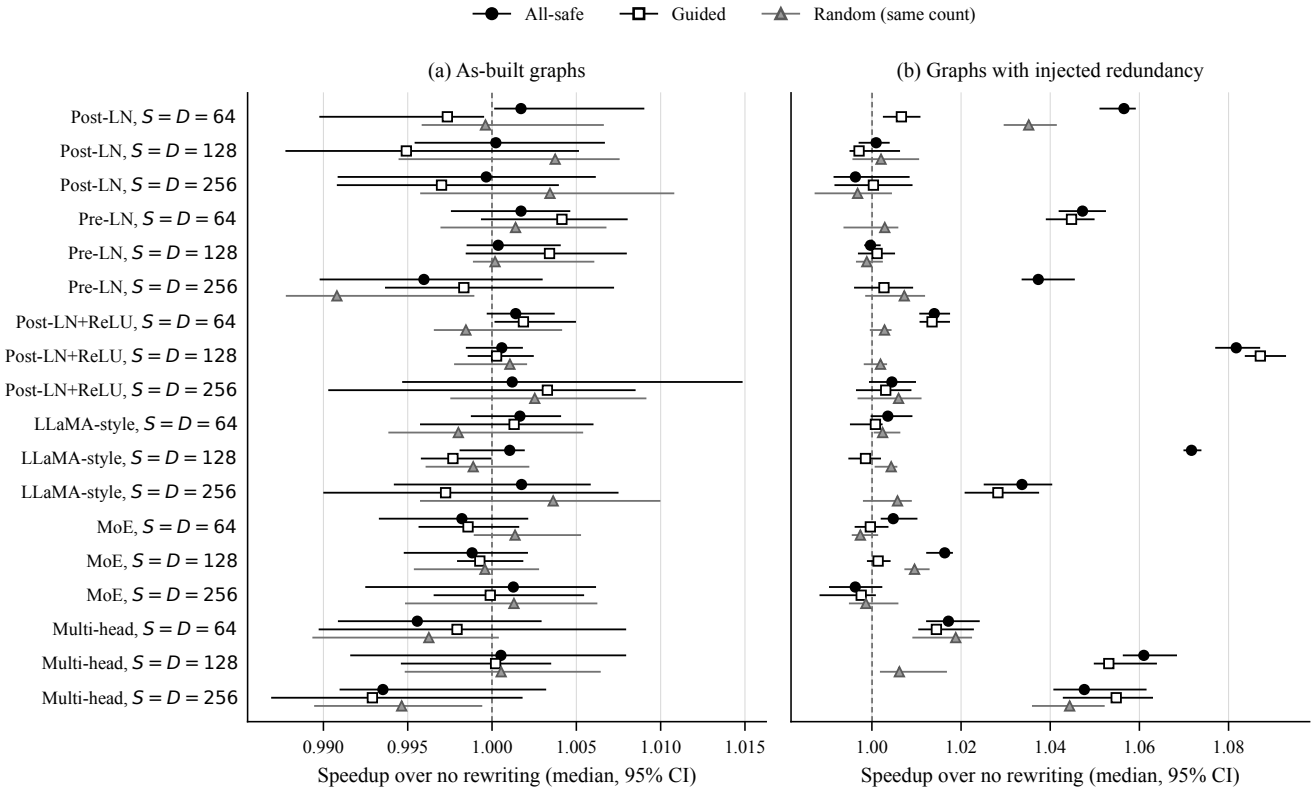


FIGURE 7. **Paired speedup over no rewriting for every workload (median of 30 rounds, 95% bootstrap interval). (a) As-built graphs; (b) graphs with injected redundancy. The two panels use different horizontal scales.**

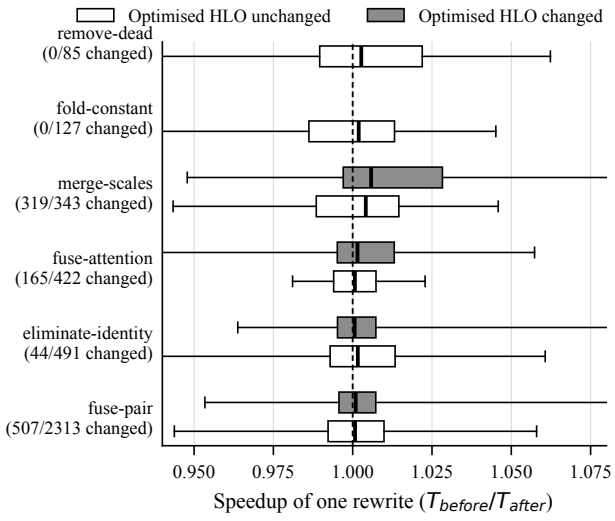


FIGURE 8. **Measured speedup of single rewrites by rule (boxes: quartiles; whiskers: 5th–95th percentiles; thick line: median), split by whether the optimized HLO changed. Labels give how many of each rule's measured rewrites changed the program.**

(15 of the 18 as-built ones), and on those 18 its measured speedups have median 1.0003 and range 0.9935–1.0017, with 17 indistinguishable from 1. This is the noise floor of the protocol. A direct example is a doubled ReLU: XLA compiles $\mathrm{ReLU}(\mathrm{ReLU}(x))$ and $\mathrm{ReLU}(x)$ to the same program, so a pre-XLA rule that removes the second ReLU cannot change execution. Rule order also matters: all-safe never applies attention fusion because its fixed order fuses the Scale+Softmax pair first, whereas guided reaches attention fusion on 17 workloads, with a small measured gain on this CPU (median 1.001).

I. GRAPH REDUCTION AND COMPILATION OVERHEAD

Table 8 reports the structural and compilation metrics per policy (medians over workloads). All-safe removes 27.8% of IR nodes and guided 20.0%, yet XLA's optimized program barely changes: the median instruction count falls from 405 to 404 and the fusion count from 49 to 48. XLA's scratch memory is identical for every policy (1,028 KiB at the median), and compile time falls by at most 3.3% because XLA receives a smaller graph. Graph reduction at the IR level therefore overstates what the compiled program gains.

J. COMPARISON WITH RELATED WORKS

Table 9 compares this work with three closely related studies of XLA and learned performance prediction (He, 2023; Phothilimthana et al., 2023; Snider & Liang, 2023). Their hardware, workloads and baselines differ from ours, so the comparison is qualitative: it contrasts what each study op-

TABLE 8. Graph reduction and compilation overhead per policy (medians over 36 workloads). Compile-time ratio is relative to the original.

Policy	Nodes	Reduction	Compile (ms)	Ratio	HLO instr. / fusions	Memory (KiB)
None	47.0	0.0%	53.4	1.000	405 / 49	1,028
All-safe	31.5	27.8%	51.4	0.967	404 / 48	1,028
Fixed (no CSE)	32.0	27.8%	51.0	0.973	404 / 48	1,028
Random (same #)	37.5	28.2%	51.6	0.991	405 / 49	1,028
Guided	37.0	20.0%	52.0	0.973	405 / 48	1,028

timizes or predicts, on which target, and what evidence it uses. This work is the only one of the four that evaluates learned selection of rewrites applied before XLA against simple baselines on the same target.

K. THREATS TO VALIDITY

Construct validity. Speedups are end-to-end latencies of single compiled calls on one input; throughput under load and energy are not measured. Rewrite correctness is checked empirically on one input per workload at a 10^{-4} relative tolerance, not proved; formal verification (Arora et al., 2025) would be stronger.

Internal validity. Single-pair labels use nine shuffled rounds and are noisy at the scale of many rewrites; the noise floor of the policy protocol is a few tenths of a percent. The guided policy uses one acceptance rule; a rule without the tie clause, or a model trained on pair labels directly, might behave differently. The latency-prior feature comes from a per-operation table calibrated on a different device; it is an input feature only, and all labels are XLA timings.

External validity. All results come from one CPU, one XLA version and six synthetic transformer architectures at small sizes, and redundancy is injected synthetically. GPUs, other XLA versions, real exported models and larger shapes may change both the absolute speedups and how much XLA already does.

Conclusion validity. We test 36 workloads with Holm correction and report interval-based verdicts per workload. The guided-versus-all-safe difference is statistically clear but small (0.13% at the median).

V. CONCLUSION AND SCOPE FOR FUTURE WORK

This work evaluated whether learned cost guidance improves the selection of output-preserving graph rewrites applied before the XLA compiler. It measured every label and every result on the same XLA target, with paired randomized timing and inspection of the compiled programs.

Existing learned cost models predict program latency from large measured datasets, and rule-based optimizers apply every legal rewrite. Such models are often trained on a different runtime and judged on whole-program latency rather than on single rewrites.

The proposed methodology trains a GNN latency model only on XLA timings and compares its guided policy with all-safe, fixed-rule, random and no rewriting. Its advantage is that it attributes a policy's result to its choices and to real compiled-program changes rather than to rewrite counts or noise.

The system was implemented in Python with JAX/XLA for compilation and PyTorch for the latency model, on one documented CPU environment.

The model ranks whole-graph latency well (Spearman $\rho = 0.91$, 20.8% error) but not single rewrites ($\rho = 0.00$), because most rewrites change latency by less than 1%. Learned guidance is 0.13% slower than all-safe ($p = 0.008$) and indistinguishable from random selection of the same number of rewrites. All policies are within 0.3% of no rewriting on graphs without injected redundancy, and only 27.4% of rewrites change the compiled program. Pre-XLA rewriting therefore pays off mainly for redundancy that XLA does not remove itself, such as chains of scaling operations.

Future work will train cost models directly on single-rewrite labels with more timing repetitions, so that the model is optimized for the decision it supports. The evaluation should be extended to GPUs, other XLA versions and real exported models at larger sizes. The acceptance rule should be revised to remove the tie clause that accepts most node-reducing rewrites. Equality-saturation search with an XLA-calibrated cost model could explore rewrite orders that greedy search cannot reach. Finally, the latency model should be validated on single-rewrite ranking under the target compiler before it is used to select rewrites.

AUTHOR CONTRIBUTIONS

All five authors contributed equally to this work: to the conceptualization and design of the study, the software, the experiments and their analysis, and the writing and revision of the manuscript.

ACKNOWLEDGMENT

The authors thank the course faculty and the open-source communities supporting the software used. AI assistants were used for code development, experiment scripting, manuscript preparation and reviewing. The authors remain responsible for reviewing the implementation, reported measurements, and claims.

FUNDING

This work received no specific funding.

COMPETING INTERESTS

The authors declare no competing interests.

DATA AND CODE AVAILABILITY

Code, raw data and scripts are available at `https://github.com/santhansai11/compiler-optimization-for-llm`, branch `integrated-final`, commit `d081a56`. The study is `benchmarks/xla_rewrite_study.py`; the results file `docs/results/xla_rewrite_study.json` contains every timing sample, every measured pair, the fold splits, the model settings and the environment;

TABLE 9. Qualitative comparison with three related studies. Hardware and baselines are not matched, so reported numbers are not directly comparable.

Aspect	He (2023)	Snider and Liang (2023)	TpuGraphs (Phothilimthana et al., 2023)	This work
Question	Where XLA succeeds and falls short for numerical models	How XLA's fusion decisions affect performance	Predicting runtime of XLA graph configurations	Whether learned guidance selects better pre-XLA rewrites
Target	XLA on CPU and GPU	XLA via JAX	XLA on TPU	XLA on CPU via JAX
Learned model	None	None	GNN ranking configurations	GNN predicting latency, trained on XLA timings
Evidence	Measured execution against reference implementations	Measured execution and inspection of fused kernels	Measured runtimes of configurations	30 paired randomized rounds, bootstrap CI, Wilcoxon, Holm; HLO fingerprints
Main finding	Gains depend on computation and hardware	Graph operation count and runtime are not monotonic	Configuration ranking is learnable but hard	Graph latency ranked well, single rewrites not; guided no better than all-safe or random

`tools/make_xla_tables.py` regenerates every table, and the optimized HLO of every variant is in `docs/results/xla_rewrite_study_hlo/`.

References

Ansel, J., Yang, E. Z., He, H., Gimelshein, N., Jain, A., Voznesensky, M., … Chintala, S. (2024). Pytorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th acm international conference on architectural support for programming languages and operating systems* (Vol. 2, pp. 929–947). Association for Computing Machinery. doi:

Arora, J., Lu, S.-H., Jain, D., Xu, T., Houshmand, F., Phothilimthana, P. M., … Mendis, C. (2025). Tensorright: Automated verification of tensor graph rewrites. *Proceedings of the ACM on Programming Languages*, *9*(POPL), 832–863. doi:

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., … Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs.* Retrieved from `https://github.com/jax-ml/jax`

Cummins, C., Wasti, B., Guo, J., Cui, B., Ansel, J., Gomez, S., … Leather, H. (2022). Compilergym: Robust, performant compiler optimization environments for AI research. In *Proceedings of the ieee/acm international symposium on code generation and optimization* (pp. 92–105). IEEE. doi:

Dao, T. (2024). Flashattention-2: Faster attention with better parallelism and work partitioning. In *International conference on learning representations.* Retrieved from `https://proceedings.iclr.cc/paper_files/paper/2024/hash/98ed250b203d1ac6b24bbcf263e3d4a7-Abstract-Conference.html`

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). Flashattention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in neural information processing systems* (Vol. 35). Retrieved from `https://arxiv.org/abs/2205.14135`

Ding, Y., Yu, C. H., Zheng, B., Liu, Y., Wang, Y., & Pekhimenko, G. (2023). Hidet: Task-mapping programming paradigm for deep learning tensor programs. In *Proceedings of the 28th acm international conference on architectural support for programming languages and operating systems* (Vol. 2, pp. 370–384). Association for Computing Machinery. doi:

Feng, S., Hou, B., Jin, H., Lin, W., Shao, J., Lai, R., … Chen, T. (2023). Tensorir: An abstraction for automatic tensorized program optimization. In *Proceedings of the 28th acm international conference on architectural support for programming languages and operating systems* (Vol. 2, pp. 804–817). Association for Computing Machinery. doi:

Gibson, P., & Cano, J. (2022). Transfer-tuning: Reusing auto-schedules for efficient tensor program code generation. In *Proceedings of the international conference on parallel architectures and compilation techniques* (pp. 28–39). Association for Computing Machinery. doi:

He, X. (2023). Accelerated linear algebra compiler for computationally efficient numerical models: Success and potential area of improvement. *PLOS ONE*, *18*(2), e0282265. doi:

Hou, Z., Liu, X., Cen, Y., Dong, Y., Yang, H., Wang, C., & Tang, J. (2022). Graphmae: Self-supervised masked graph autoencoders. In *Proceedings of the 28th acm sigkdd conference on knowledge discovery and data mining* (pp. 594–604). Association for Computing Machinery. doi:

Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J., … Zinenko, O. (2021). MLIR: Scaling compiler infrastructure for domain specific computation. In *Proceedings of the ieee/acm international symposium on code generation and optimization* (pp. 2–14). IEEE. doi:

Li, M., Liu, Y., Liu, X., Sun, Q., You, X., Yang, H., … Qian, D. (2021). The deep learning compiler: A comprehensive survey. *IEEE Transactions on Parallel and Distributed Systems*, *32*(3), 708–727. doi:

Niu, W., Guan, J., Wang, Y., Agrawal, G., & Ren, B. (2021).

Dnnfusion: Accelerating deep neural networks execution with advanced operator fusion. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation* (pp. 883–898). Association for Computing Machinery. doi:

Phothilimthana, P. M., Abu-El-Haija, S., Cao, K., Fatemi, B., Burrows, M., Mendis, C., & Perozzi, B. (2023). Tpugraphs: A performance prediction dataset on large tensor computational graphs. In *Advances in neural information processing systems* (Vol. 36, pp. 70355–70375). Curran Associates, Inc. doi:

Ryu, J., Park, E., & Sung, H. (2022). One-shot tuner for deep learning compilers. In *Proceedings of the 31st acm sigplan international conference on compiler construction* (pp. 89–103). Association for Computing Machinery. doi:

Shao, J., Zhou, X., Feng, S., Hou, B., Lai, R., Jin, H., … Chen, T. (2022). Tensor program optimization with probabilistic programs. In *Advances in neural information processing systems* (Vol. 35, pp. 35783–35796). doi:

Shen, C., Shen, Y., Xu, T., Guo, B., Jiang, L., Zhang, M., … Pimentel, A. D. (2026). Graph-based clustering cost model for performance optimization in deep learning compilers. *Journal of Systems Architecture*, *178*, 103895. doi:

Shi, Y., Yang, Z., Xue, J., Ma, L., Xia, Y., Miao, Z., … Zhou, L. (2023). Welder: Scheduling deep learning memory access via tile-graph. In *Proceedings of the 17th usenix symposium on operating systems design and implementation* (pp. 701–718). USENIX Association. Retrieved from https://www.usenix.org/conference/osdi23/presentation/shi

Snider, D., & Liang, R. (2023). *Operator fusion in XLA: Analysis and evaluation.* Retrieved from https://arxiv.org/abs/2301.13062

Trofin, M., Qian, Y., Brevdo, E., Lin, Z., Choromanski, K., & Li, D. (2021). *MLGO: A machine learning guided compiler optimizations framework.* Retrieved from https://arxiv.org/abs/2101.04808

Unger, C., Jia, Z., Wu, W., Lin, S., Baines, M., Quintero Narvaez, C. E., … Aiken, A. (2022). Unity: Accelerating DNN training through joint optimization of algebraic transformations and parallelization. In *Proceedings of the 16th usenix symposium on operating systems design and implementation* (pp. 267–284). USENIX Association.

Wang, H., Zhai, J., Gao, M., Ma, Z., Tang, S., Zheng, L., … Jia, Z. (2021). PET: Optimizing tensor programs with partially equivalent transformations and automated corrections. In *Proceedings of the 15th usenix symposium on operating systems design and implementation* (pp. 37–54). USENIX Association.

Willsey, M., Nandi, C., Wang, Y. R., Flatt, O., Tatlock, Z., & Panchekha, P. (2021). egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, *5*(POPL), 1–29. doi:

Yang, Y., Phothilimthana, P. M., Wang, Y. R., Willsey, M., Roy, S., & Pienaar, J. (2021). Equality saturation for tensor graph superoptimization. In *Proceedings of machine learning and systems* (Vol. 3, pp. 255–268). Retrieved from https://proceedings.mlsys.org/paper_files/paper/2021/hash/cc427d934a7f6c0663e5923f49eba531-Abstract.html

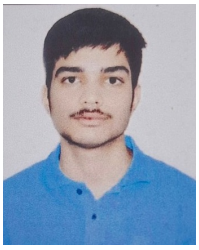
Zhai, Y., Zhang, Y., Liu, S., Chu, X., Peng, J., Ji, J., & Zhang, Y. (2022). *TLP: A deep learning-based cost model for tensor program tuning.* Retrieved from https://arxiv.org/abs/2211.03578

Zhang, S., Li, M., Yang, H., Liu, Y., Luan, Z., & Qian, D. (2022). *FamilySeer: Towards optimized tensor codes by exploiting computation subgraph similarity.* Retrieved from https://arxiv.org/abs/2201.00194

Zheng, B., Jiang, Z., Yu, C. H., Shen, H., Fromm, J., Liu, Y., … Pekhimenko, G. (2022). DietCode: Automatic optimization for dynamic tensor programs. In *Proceedings of machine learning and systems* (Vol. 4, pp. 848–863). Retrieved from https://proceedings.mlsys.org/paper/2022/file/fa7cdfad1a5aaf8370ebeda47a1ff1c3-Paper.pdf

Zheng, L., Liu, R., Shao, J., Chen, T., Gonzalez, J. E., Stoica, I., & Haj-Ali, A. (2021). TenSet: A large-scale program performance dataset for learned tensor compilers. In *Proceedings of the neural information processing systems track on datasets and benchmarks* (Vol. 1). Retrieved from https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/a684eceee76fc522773286a895bc8436-Abstract-round1.html

Zhu, H., Wu, R., Diao, Y., Ke, S., Li, H., Zhang, C., … Pekhimenko, G. (2022). ROLLER: Fast and efficient tensor compilation for deep learning. In *Proceedings of the 16th usenix symposium on operating systems design and implementation* (pp. 233–248). USENIX Association. Retrieved from https://www.usenix.org/conference/osdi22/presentation/zhu



SHWETANK SHRIJAN is a B.Tech student at Vellore Institute of Technology, Vellore, in the Department of Computer Science and Engineering (Data Science).



BHUVANESWARI M is with the Department of Computer Science, Vellore Institute of Technology, Vellore, India.



SARVESH SUHAS KUTE is currently pursuing a B.Tech in Computer Science and Engineering at Vellore Institute of Technology (VIT), Vellore, India. He completed a software engineering internship at Supertech Instrumentation Services, where he built REST APIs and a React dashboard for industrial plant monitoring and reduced database query latency by 85%. His research interests include AI infrastructure, distributed systems, operating systems, and agentic AI. He has developed an eBPF-based Linux file prefetcher and a concurrent network service platform.



THOTA PENCHALA SANTHAN SAI is currently pursuing a B.Tech in Computer Science and Engineering at Vellore Institute of Technology (VIT), Vellore, India. His research interests include compiler optimization for large language models (LLMs), AI infrastructure, efficient model inference, cloud computing, and DevOps. He is particularly interested in compiler-level techniques for improving LLM execution performance, reducing computational overhead, and optimizing resource utilization. He has worked on projects involving AI-based emotion recognition, web application development, and cloud infrastructure.



VIDHUR VAIBHAV B S is an undergraduate student pursuing a degree in B.Tech Computer Science and Engineering at Vellore Institute of Technology. His academic interests include machine learning and LLMs, with a particular focus on compiler technologies and their optimisations. This paper examines whether a graph neural network trained on XLA-compiled timings works with similar policies across transformer workloads.